Домашнее задание № 4

Обеспечение отказоустойчивости средствами Apache ZooKeeper

Расчёт пар навыков в вакансиях DevOps

Смирнов Александр, группа J3310

30 октября 2026 года

Цель работы

Развернуть ансамбль ZooKeeper из трёх серверов и проверить кворум и эфемерные узлы. Интегрировать его с расчётом пар навыков из ДЗ № 3: выбрать активный координатор, восстановить расчёт после его приостановки и сравнить публикацию результата без защиты и с проверкой эпохи.

Основные результаты

Проверка	Результат
Ансамбль	Один лидер и два последователя; один отказ допустим, два приводят к потере кворума
Эфемерный узел	Создан через zkCli и удалён после закрытия сессии
Восстановление	Запасной загрузил 1 готовый чанк с 218 вакансиями и обработал оставшиеся 28 чанков
Публикация	Без защиты приняты две записи; с защитой старая эпоха отклонена с BadVersionError
Корректность	Все четыре полных результата совпали с эталоном ДЗ № 3 по SHA-256

Стенд и конфигурация

Стенд запущен в Docker Desktop с Linux-контейнерами. Использован официальный образ ZooKeeper 3.9.6 с Java 17. Каждый сервер имеет отдельные тома /data и /datalog; одинаковые пути внутри контейнеров изолированы. Общая конфигурация config/zoo.cfg задаёт один и тот же состав ансамбля, а ZOO_MY_ID создаёт разный файл /data/myid.

Сервер	myid	Клиентский порт	Начальная роль
zk1	1	127.0.0.1:2181	follower
zk2	2	127.0.0.1:2182	follower
zk3	3	127.0.0.1:2183	leader

```
tickTime=2000
initLimit=10
syncLimit=5
dataDir=/data
dataLogDir=/datalog
clientPort=2181
minSessionTimeout=4000
maxSessionTimeout=20000
standaloneEnabled=false
server.1=zk1:2888:3888
server.2=zk2:2888:3888
server.3=zk3:2888:3888
```

Фрагмент zoo.cfg. Порты 2888 и 3888 используются внутри сети Compose. Клиентские порты опубликованы только на localhost.

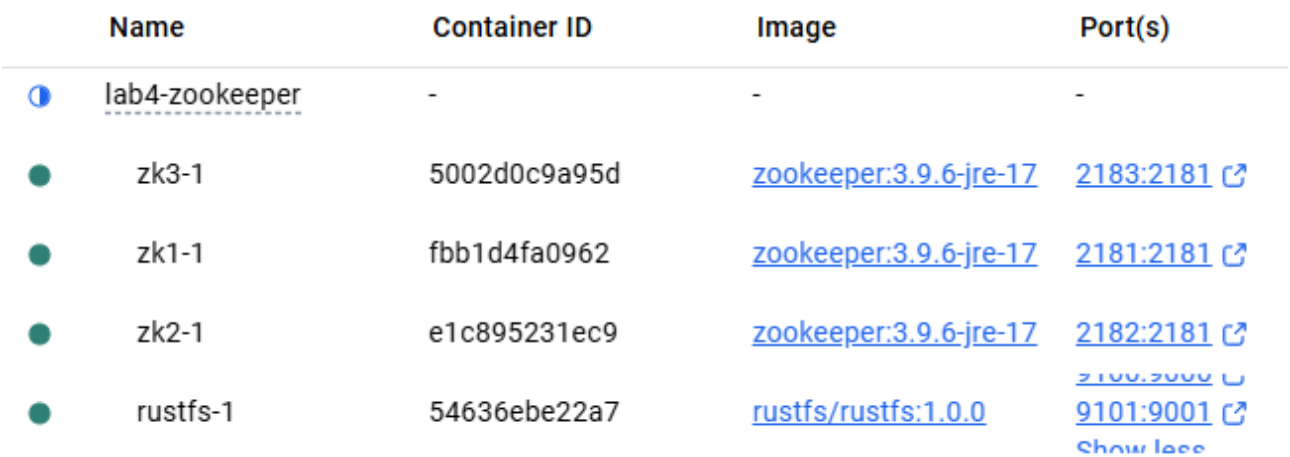


Рисунок 1. Три контейнера ZooKeeper 3.9.6 и отдельный RustFS. Снимок показывает состояние стенда после экспериментов; роли подтверждены ответами srvr.

Для приложения используются два координатора и отдельный получатель публикаций. RustFS ДЗ № 4 доступен на портах 9100 и 9101 и не пересекается с хранилищем ДЗ № 3.

Проверка кворума и эфемерного узла

```
python scripts/check_ensemble.py
```

Сценарий определяет роли через TCP-команду srvr, затем проверяет реальную запись и чтение контрольного значения. В начальных ответах zk1 и zk2 указано Mode: follower, у zk3 - Mode: leader.

Состояние	Наблюдение
Работают все три сервера	Один лидер, два последователя; запись и чтение успешны
Остановлен zk1	Запись нового значения и чтение через оставшийся кворум успешны
Остановлены zk1 и zk2	Read-write сессия не устанавливается; ответ zk3 приведён ниже
zk1 и zk2 возвращены	Снова один лидер и два последователя; запись и чтение восстановлены

```
$ docker compose stop zk2
 Container lab4-zookeeper-zk2-1  Stopping
 Container lab4-zookeeper-zk2-1  Stopped
```

```
This ZooKeeper instance is not currently serving requests
```

Контейнер zk3 продолжает работать, но без большинства не обслуживает запросы. Проверка не разрешает read-only соединение; ожидание обычной сессии завершилось по тайм-ауту 5 секунд.

Эфемерный узел

Узел создан штатным zkCli.sh. Независимый клиент Kazoo подтвердил его наличие и ненулевой ephemeralOwner, затем дождался исчезновения после quit.

```
$ docker compose exec -T zk1 zkCli.sh -server zk2:2181
> create -e /lab4/ensemble/ephemeral-1793355661669150900 session-owned
> quit
[zk: zk2:2181(CONNECTED) 0] Created /lab4/ensemble/ephemeral-1793355661669150900
WatchedEvent state:Closed type:None path:null zxid: -1
```

Проверка наблюдателя	Значение из summary.json
Владелец сессии	0x2000001ab150000
Узел отсутствует после quit	true
Все проверки ансамбля выполнены	true

Эфемерный узел связан с сессией. При quit она закрывается явно; при зависании процесса узел удаляется после истечения сессии. Именно истечение сессии далее освобождает лидерство координатора.

Координатор и восстановление из чекпоинта

Оба координатора используют один NDJSON-сегмент ДЗ № 3: 6 215 вакансий, 1 880 228 байт. Сохранены правила S3 Range GET, нормализация навыков и формат pairs.json. Два Mapper-процесса считают пары по чанкам размером 65 536 байт, координатор объединяет частоты.

Состояние	Место и назначение
election/candidate-...	Эфемерные последовательные узлы; минимальный номер выбирает лидера
epoch	Версия полномочий; повышается новым лидером
checkpoint	Указатель на S3-объект со счётчиками, числом вакансий и ID готовых чанков
result, publications	Указатель полного результата и журнал принятых публикаций

Запасной проверяет SHA-256 чекпоинта, идентификатор входа и размер чанка, восстанавливает Counter и отправляет Mapper только оставшиеся задачи. Готовые чанки не учитываются повторно. В обоих опытах журнал запасного содержит checkpoint_loaded: completed=1, vacancies=218. Он продолжил расчёт с 28 оставшихся чанков, а не начал с нуля.

ZooKeeper хранит небольшие указатели, крупные счётчики остаются в RustFS. Получатель публикаций работает с собственной активной сессией. Проверка эпохи чекпоинта включена в обоих режимах; FENCING меняет только защиту финальной публикации.

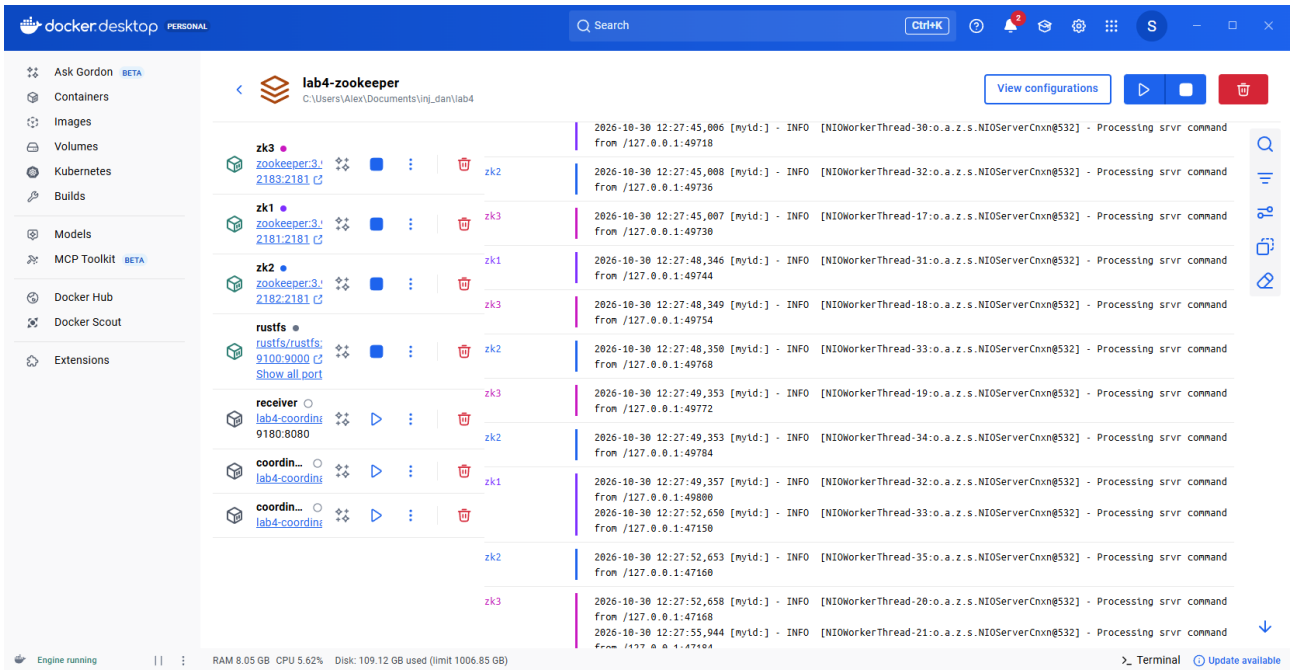


Рисунок 2. Проект lab4-zookeeper и текущие журналы в Docker Desktop. Координаторы и получатель после опытов остановлены. События экспериментов подтверждены сохранёнными журналами в следующих разделах.

Приостановка лидера без защиты

```
python scripts/run_failover.py --fencing 0
```

После первого чекпоинта coordinator-a приостановлен целиком через docker compose pause минимум на 40 секунд. Тайм-аут сессии равен 10 секундам. После её истечения coordinator-b получил эпоху 2, восстановил чекпоинт и опубликовал результат. Старый лидер возобновлён после этой публикации.

Время журнала, UTC	Процесс	Событие
10:27:59.402	coordinator-a	Получил лидерство, эпоха 1
10:28:10.453	coordinator-b	Получил лидерство, эпоха 2
10:28:10.526	coordinator-b	Загрузил 1 чанк, 218 вакансий
10:28:19.041	coordinator-b	Записал результат, версия 1
10:28:47.810	coordinator-a	Записал результат, версия 2

Две записи «записал результат»

Из receiver.jsonl выбраны события публикации; поля key и component опущены.


```
{
  "time": "2026-10-30T10:28:19.041298+00:00",
  "event": "записал результат",
  "coordinator": "coordinator-b",
  "epoch": 2,
  "version": 1
}

{
  "time": "2026-10-30T10:28:47.810280+00:00",
  "event": "записал результат",
  "coordinator": "coordinator-a",
  "epoch": 1,
  "version": 2
}
```


Итог	Значение
Время до обнаружения запасного лидером	9,717 с
Принято публикаций / версия result	2 / 2
Итоговый указатель	coordinator-a, эпоха 1

Старый лидер завершил вычисление и был принят вторым с устаревшей эпохой 1. Численные результаты совпали, но двойная публикация состоялась. Для приложения с побочными эффектами она могла бы иметь последствия.

Приостановка лидера с защитой по эпохе

```
python scripts/run_failover.py --fencing 1
```

Опыт повторён с тем же входом и параметрами в отдельном пространстве ZooKeeper и префиксе S3. Получатель проверяет текущую эпоху и меняет указатель одной транзакцией. Фрагмент src/receiver.py:

```
transaction = zk.transaction()
transaction.check(root + '/epoch', version=request['epoch'])
transaction.set_data(root + '/result', encode(request), version=-1)
transaction.create(root + '/publications/entry-',
                   encode(request), sequence=True)
commit_checked(transaction)
```

Время журнала, UTC	Процесс	Событие
10:29:05.786	coordinator-a	Получил лидерство, эпоха 1
10:29:17.552	coordinator-b	Получил лидерство, эпоха 2
10:29:17.618	coordinator-b	Загрузил 1 чанк, 218 вакансий
10:29:28.753	coordinator-b	Записал результат, версия 1
10:29:56.626	coordinator-a	BadVersionError

Принятая и отклонённая публикации


```
{
  "time": "2026-10-30T10:29:28.753034+00:00",
  "event": "записал результат",
  "coordinator": "coordinator-b",
  "epoch": 2,
  "version": 1
}

{
  "time": "2026-10-30T10:29:56.626173+00:00",
  "event": "publication_rejected",
  "coordinator": "coordinator-a",
  "epoch": 1,
  "error": "BadVersionError"
}
```


Итог	Значение
Время до обнаружения запасного лидером	10,320 с
Принято / отклонено публикаций	1 / 1
Версия и владелец result	1; coordinator-b, эпоха 2

BadVersionError не даёт старому координатору изменить указатель. В S3 может остаться его неизменяемая заготовка, но опубликованным считается принятый result и запись publications: с защитой такая публикация одна.

Проверка результатов и выводы

Показатель	Без защиты	С защитой
Время смены лидера	9,717 с	10,320 с
Загружено из чекпоинта	1 чанк, 218 вакансий	1 чанк, 218 вакансий
Принято публикаций / версия result	2 / 2	1 / 1
Устаревшая публикация	Принята	BadVersionError
Полное совпадение с ДЗ № 3	Оба координатора	Оба координатора

Во всех четырёх результатах 6 215 вакансий, 25 462 уникальные пары и 173 204 вхождения. Совпал SHA-256 полного канонического JSON, включая метаданные и все частоты:

```
cafaec2c23af084d67ce40c8cf4e7847ab46bf54766cb1663d1213d500877a12
```

Как измерено время

Используются монотонные часы управляющего скрипта: от подтверждения pause до обнаружения новой эпохи, с опросом каждые 0,1 секунды. Отметки UTC Windows и контейнеров не вычитаются друг из друга. Интервал включает задержку наблюдения и зависит от последнего heartbeat и момента истечения сессии; тайм-аут 10 секунд не означает ровно 10 секунд после pause.

Почему три сервера, а не два или четыре

Для двух серверов большинство равно двум: один отказ останавливает ансамбль. У трёх нужны два голоса, поэтому один отказ допустим. У четырёх нужны три, и допустим всё тот же один отказ. Три сервера дают нужную устойчивость без четвёртого участника, который её не увеличивает; для двух отказов нужны пять.

Почему эпоху проверяет получатель

ZooKeeper завершает сессию, но не может остановить старый процесс и отменить его запрос к другому сервису. Поэтому получатель должен отклонить запись с устаревшими полномочиями. В нашей реализации приложение требует проверки версии, а ZooKeeper атомарно выполняет её вместе с обновлением указателя. Отдельная проверка до записи оставила бы промежуток для смены лидера.

Кворум сохраняет работоспособность координации при одном отказе; чекпоинт сохраняет выполненную часть расчёта; проверка эпохи защищает публикацию после возвращения старого лидера. Это три дополняющих друг друга механизма.

Повторение опыта и материалы

```
pip install -r requirements.txt
docker compose up -d --wait
python scripts/check_ensemble.py
python scripts/prepare_data.py
python -m unittest discover -s tests -v
python scripts/run_failover.py --fencing 0
python scripts/run_failover.py --fencing 1
```

Команды выполняются в Python-окружении проекта. prepare_data.py берёт NDJSON из соседнего lab3; другой путь задаётся через --source. Коммит и хеш входа закреплены в reference/lab3.json. Каждый опыт создаёт свой run_id.

Материал	**Подтверждение**
compose.yaml; config/zoo.cfg	Три участника, myid и раздельное хранение
logs/ensemble/	Роли, потеря одного и двух серверов, создание и исчезновение эфемерного узла
fencing-0-20261030T102741-190900	Две публикации, указатель версии 2 и измерение смены лидера
fencing-1-20261030T102900-401400	BadVersionError, указатель запасного версии 1 и измерение смены лидера
summary.json обоих запусков	Чекпоинт, четыре хеша результатов и all_passed=true

Каталоги запусков находятся в logs/. В каждом сохранены coordinator-a.jsonl, coordinator-b.jsonl, receiver.jsonl, commands.log и summary.json. Снимки находятся в report/figures/. Полные журналы не заменены картинками.

Границы проверки

Все контейнеры размещены на одном компьютере: проверены отказы процессов, а не независимых машин. Каждый режим выполнен одним полным запуском. Отказ RustFS и самого получателя, повреждение диска и обрыв ответа после успешной записи в этих опытах не моделировались.

Источники

Код и журналы: https://github.com/alexsmirnov6/lab4_zookeeper
Эталон ДЗ № 3: https://github.com/alexsmirnov6/lab3_devops

Apache ZooKeeper 3.9.6: конфигурация и кворум.
https://zookeeper.apache.org/doc/r3.9.6/zookeeperAdmin.html

Apache ZooKeeper 3.9.6: сессии, эфемерные узлы и версии.
https://zookeeper.apache.org/doc/r3.9.6/zookeeperProgrammers.html

Образ: https://hub.docker.com/_/zookeeper
Клиент и транзакции: https://kazoo.readthedocs.io/en/latest/api/client.html